

CONTROL DE INVENTARIO

Bitácora técnica y estado del proyecto

Documento de continuidad del trabajo

Corte al 3 de octubre de 2026

Resumen del esquema de base de datos, seguridad, funciones de inventario, cambios ejecutados, verificaciones y próximos pasos.

Estado general: estructura inicial aplicada; funciones principales instaladas; pruebas funcionales aún pendientes.

1. Objetivo y alcance

Este documento consolida el trabajo realizado en el proyecto Control de inventario hasta la fecha de corte. Está pensado como bitácora de continuidad para retomar la implementación sin repetir verificaciones que ya se completaron.

El sistema se está construyendo para manejar inventario por empresa y almacén, con productos, movimientos, existencias, costos y control de acceso por usuario y rol.

2. Entorno de desarrollo

Elemento	Detalle
Proyecto	C:\Repos\projects\inventario-universal
Tecnología	Next.js, TypeScript, Tailwind CSS, Supabase/PostgreSQL
Supabase local	API: http://127.0.0.1:54321
Studio local	http://127.0.0.1:54323
Puerto PostgreSQL	54322
Comando de desarrollo	npm.cmd run dev (en PowerShell, npm.ps1 puede estar bloqueado por la política de ejecución)
Migración inicial	supabase/migrations/20261003201204_initial_inventory_schema.sql
Respaldo SQL	supabase/20261003201204_initial_inventory_schema.backup.sql

La configuración local, el archivo .env.local y las credenciales no se incluyen en este PDF. El archivo .env.local está excluido de Git.

3. Modelo de datos

Tabla	Responsabilidad
businesses	Empresas o negocios aislados dentro de la aplicación.
business_members	Relación usuario-empresa y rol asignado.
warehouses	Almacenes pertenecientes a una empresa.
products	Catálogo de productos de cada empresa.
inventory_balances	Existencia y costo promedio por empresa, almacén y producto.
inventory_movements	Cabecera del movimiento, tipo, estado, motivo y trazabilidad de creación/confirmación.
inventory_movement_lines	Productos afectados por el movimiento, cantidades, conteos físicos y datos de trazabilidad.

Restricciones importantes verificadas

- Las empresas y los almacenes se relacionan mediante claves foráneas y claves compuestas para evitar cruces entre empresas.
- SKU único dentro de cada empresa; código de almacén único dentro de cada empresa.
- Cantidades de líneas mayores que cero; costos y existencias no negativos; conteo físico no negativo.
- Roles permitidos: owner, admin, operator y viewer.
- Tipos de movimiento: receipt, issue y adjustment; estados: draft, posted y cancelled.
- Una reversión por movimiento original mediante la restricción única sobre reversal_of.

4. Seguridad y permisos

Row Level Security (RLS)

Las políticas verificadas permiten lectura a usuarios autenticados según su pertenencia a la empresa. Para empresas y miembros se usa la función `is_business_member(uuid)`; las tablas de almacenes, productos, existencias y movimientos filtran por pertenencia a la empresa.

La función `public.is_business_member(p_business_id uuid)` es SECURITY DEFINER, propiedad de postgres, con `search_path` vacío y ejecución concedida a `authenticated`, no a `anon`.

Permisos de tablas

Se verificó que `authenticated` tiene permisos SELECT en las siete tablas. No se concedieron permisos directos de escritura a `authenticated` en las tablas de inventario; las operaciones se canalizan mediante funciones controladas. `anon` no tiene permisos directos de tabla según la verificación previa.

Funciones SECURITY DEFINER verificadas

Función	authenticated	anon	Propietario
<code>create_inventory_movement</code>	EXECUTE	Sin EXECUTE	postgres
<code>add_inventory_movement_line</code>	EXECUTE	Sin EXECUTE	postgres
<code>post_inventory_movement</code>	EXECUTE	Sin EXECUTE	postgres
<code>reverse_inventory_movement</code>	EXECUTE	Sin EXECUTE	postgres

Los resultados confirmaron SECURITY DEFINER = true para las cuatro funciones, ejecución permitida a `authenticated` y denegada a `anon`.

Roles previstos

Rol	Alcance funcional previsto
owner / admin	Lectura, catálogo de productos, almacenes, movimientos y administración de miembros.
operator	Lectura y registro de movimientos; sin administrar productos, almacenes ni miembros.
viewer	Solo lectura.

La matriz anterior representa el diseño previsto; aún falta completar y probar todos los controles de autorización, especialmente la inicialización de empresas y miembros.

5. Funciones de inventario instaladas

`create_inventory_movement(...)`

Crea una cabecera de movimiento en borrador. Verifica que haya sesión, que el tipo sea válido, que el motivo no esté vacío, que la empresa esté activa, que el usuario tenga rol owner/admin/operator y que el almacén pertenezca a la empresa y esté activo.

`add_inventory_movement_line(...)`

Agrega una línea a un movimiento en borrador, valida permisos, pertenencia de empresa y producto activo, y valida campos de acuerdo con el tipo de movimiento. Las entradas requieren cantidad positiva y costo unitario; las salidas requieren cantidad positiva; los ajustes usan conteo físico.

`post_inventory_movement(uuid)`

Confirma el movimiento dentro de una transacción. Valida usuario, rol, estado de borrador, empresa y almacén activos, existencia de líneas y productos activos. Bloquea filas de existencias para evitar actualizaciones simultáneas inconsistentes.

- Entrada (receipt): suma existencias y calcula el costo promedio ponderado.
- Salida (issue): rechaza existencias insuficientes, resta cantidad y conserva el costo promedio.
- Ajuste (adjustment): compara conteo físico con existencia teórica y registra incremento o disminución; conserva el costo promedio.
- Registra `stock_before`, `stock_after` y `applied_unit_cost` para trazabilidad.
- Confirma el movimiento con `status=posted`, `posted_by` y `posted_at`.

Corrección aplicada

Se reemplazó la función para eliminar las líneas de ajuste cuya diferencia entre conteo físico y existencia teórica sea cero. Si todas las líneas de un ajuste carecen de diferencia, la función genera una excepción; por la atomicidad transaccional, no deben persistir los cambios del intento y el movimiento debe permanecer en borrador.

`reverse_inventory_movement(p_movement_id uuid, p_reason text)`

Crea un movimiento inverso para un movimiento confirmado, exige motivo y rol autorizado, bloquea el movimiento original y evita una segunda reversión. Entrada se revierte con salida; salida con entrada; ajuste con otro ajuste que restaura el conteo anterior cuando se cumplen las validaciones.

Limitaciones y precauciones conocidas

- Revertir una entrada requiere existencias suficientes; la salida inversa conserva el costo promedio vigente, no necesariamente el costo histórico exacto.
- Revertir una salida crea una entrada al costo aplicado histórico y recalcula el costo promedio ponderado.
- La reversión de ajustes exige que la existencia actual coincida con `stock_after` registrado.
- Movimientos antiguos sin datos de trazabilidad suficientes pueden rechazarse para evitar una reversión insegura.
- Las funciones se instalaron, pero su comportamiento completo aún no se ha demostrado mediante pruebas funcionales.

6. Triggers y trazabilidad

Tabla	Trigger	Estado observado
inventory_movement_lines	inventory_movement_lines_guard	O — habilitado
inventory_movements	inventory_movements_changes_guard	O — habilitado

Tabla	Trigger	Estado observado
inventory_movements	inventory_movements_status_guard	0 — habilitado

Los triggers protegen las líneas para que solo puedan modificarse mientras el movimiento está en borrador, evitan borrar físicamente movimientos y controlan transiciones de estado. Se verificó que los tres están habilitados.

Columnas de trazabilidad añadidas a inventory_movement_lines

Columna	Uso
stock_before	Existencia antes de aplicar la línea.
stock_after	Existencia resultante después de aplicar la línea.
applied_unit_cost	Costo unitario aplicado para documentar el movimiento.

Se confirmó su existencia y tipo numeric.

7. Estado de la base de datos

La última consulta de conteos devolvió cero registros en todas las tablas principales:

Tabla / concepto	Conteo
Empresas	0
Miembros	0
Almacenes	0
Productos	0
Movimientos	0
Líneas de movimientos	0
Existencias	0

Esto significa que no hay datos de negocio que puedan alterarse durante pruebas iniciales. También significa que todavía no se ha probado el flujo real de inventario.

8. Pendientes identificados

1. Resolver la inicialización segura de una empresa y su propietario autenticado. No se encontró una función específica para crear empresas ni para asociar miembros.
2. Revisar estructura, columnas, valores por defecto, restricciones y permisos de escritura de businesses y business_members. La última consulta solicitada para ello aún no tiene respuesta en el historial.
3. Implementar o definir el flujo de alta de empresa que asigne al usuario autenticado como owner sin insertar usuarios ficticios ni confiar en datos proporcionados por el cliente.
4. Completar los controles de rol en todas las operaciones, incluida administración de productos, almacenes y miembros.
5. Ejecutar pruebas integrales de entradas, salidas, ajustes con diferencia, ajustes mixtos, ajustes sin diferencia, existencias insuficientes y reversión.
6. Probar rechazo de operaciones no autorizadas y aislamiento entre empresas.
7. Verificar atomicidad: una operación fallida no debe dejar balances, líneas ni estados parcialmente modificados.
8. Revisar la reversión en presencia de movimientos posteriores y confirmar el comportamiento de costos promedio con casos controlados.
9. Solo después de superar pruebas, integrar el flujo con la interfaz Next.js y considerar datos reales.

9. Plan de pruebas propuesto

Prueba	Resultado esperado
Entrada de 10 unidades a costo 20	Existencia +10; costo promedio 20.
Entrada adicional de 10 a costo 30	Existencia 20; costo promedio ponderado 25.
Salida de 4 unidades	Existencia 16; costo promedio permanece 25.
Salida superior a existencias	Error; sin cambios parciales.
Ajuste de existencia 16 a conteo 13	Existencia 13; línea de disminución de 3.
Ajuste mixto con una línea sin diferencia	La línea sin cambio no permanece en el movimiento confirmado.
Ajuste sin ninguna diferencia	Error; movimiento sigue en borrador y no cambia existencias.
Reversión de movimiento	Se crea un movimiento inverso, sujeto a reglas de stock y trazabilidad.
Acceso de viewer a escritura	Operación denegada.
Usuario de otra empresa	Sin acceso a los datos de la empresa ajena.

Las cifras anteriores son escenarios de prueba propuestos, no resultados observados.

10. Historial resumido de cambios

- Se creó y aplicó la migración inicial del esquema de inventario.
- Se revisaron restricciones, permisos, políticas RLS y triggers.
- Se instalaron las funciones de creación de movimientos, alta de líneas, confirmación y reversión.
- Se añadieron columnas de trazabilidad a las líneas.
- Se corrigió post_inventory_movement para retirar líneas de ajustes sin diferencia.
- Se verificaron permisos de ejecución: authenticated puede ejecutar las cuatro funciones; anon no.
- Se confirmó que los tres triggers están habilitados.
- Se confirmó que la base de datos de negocio permanece vacía.

11. Punto exacto para retomar

La próxima acción pendiente es revisar las columnas y los permisos de businesses y business_members. Después se debe diseñar el alta segura de empresa/propietario, preparar datos de prueba aislados y ejecutar el plan funcional. No se ha completado todavía una validación integral ni se debe considerar el sistema listo para producción.

Nota de fidelidad

Este PDF resume lo que se ha compartido y verificado en la conversación. No sustituye la migración SQL ni una copia de seguridad de la base de datos. La definición completa de post_inventory_movement fue revisada durante la conversación; para recuperar otras funciones completas debe consultarse la base local o la migración correspondiente.